iterating toward a clean, maintainable solution.

Answer: A custom React Hook like `useDebounce` encapsulates reusable logic to delay updates to a value until the user stops typing or interacting for a specified duration. This is useful in scenarios where you want to avoid excessive function calls—for example, triggering API requests during rapid input changes.

TIP

During an interview, before diving into implementation, it is essential to clarify the requirements with the interviewer. Take a moment to ask questions and gather detailed requirements. This ensures you understand the task accurately and can deliver a solution that meets the interviewer's expectations.

Table 4.8 are example requirements after clarified with the interviewer.

Table 4.8 Requirements for implementing the useDebounce hook

Aspect	Description
Hook purpose	Delay updates to a value until a specified timeout period after the last change.
Use case	Ideal for search bars, autocomplete fields, or resize listeners to prevent unnecessary re-renders or API calls.
Input arguments	Input arguments: `value` (the value to debounce) and `delay` (the debounce delay in milliseconds, defaulting to 500ms).
Return value	The debounced value after the timeout period.
Design considerations	The value argument should support generic types (`T`) to make it reusable for any data type. Use the `useEffect` hook with `setTimeout` to control the delay.